Aula 17 – Strings e Bibliotecas em C

1. Objetivos de aprendizagem

- Compreender a finalidade da biblioteca <string.h>.
- Utilizar strlen() para obter o comprimento de uma string.
- Comparar strings com strcmp().
- Copiar strings utilizando strcpy().
- Concatenar strings com strcat().
- Aplicar essas funções na resolução de problemas práticos.

2. Relembrando a aula anterior

Na Aula 16, aprendemos que uma string é um vetor de caracteres terminado pelo caractere especial '\0'. Também desenvolvemos um programa que percorria uma palavra para contar seus caracteres:

```
char palavra[] = "CASA";
int contador = 0;

for (int i = 0; palavra[i] != '\0'; i++) {
   contador++;
}

printf("Quantidade: %d\n", contador);
```

Será que precisamos criar um laço de repetição toda vez que quisermos contar os caracteres de uma palavra? Não. A linguagem C disponibiliza funções que facilitam esse trabalho.

3. O que são bibliotecas em C?

Bibliotecas são conjuntos de funções prontas que podem ser utilizadas em nossos programas. Já utilizamos a biblioteca <stdio.h> para operações de entrada e saída, como printf() e scanf().

Agora conheceremos a biblioteca <string.h>, utilizada para manipular strings.

Função	Finalidade
`strlen()`	Contar caracteres de uma string
`strcmp()`	Comparar duas strings
`strcpy()`	Copiar uma string para outra
`strcat()`	Juntar duas strings

4. Função strlen() – Comprimento de uma string

Imagine um sistema de cadastro que precisa verificar quantos caracteres existem no nome de um usuário.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char nome[50];

    printf("Digite seu primeiro nome: ");
    scanf("%49s", nome);

    printf("Quantidade de caracteres: %zu\n", strlen(nome));

    return 0;
}
```

Exemplo de saída:

Digite seu primeiro nome: Mariana
Quantidade de caracteres: 7

A função strlen() conta os caracteres até encontrar '\0', sem incluir esse terminador na contagem. O especificador %zu é utilizado para exibir o valor retornado pela função.

5. Função strcmp() – Comparação de strings

Imagine um sistema que precisa verificar se o usuário digitou a palavra ADMIN.

```
#include <stdio.h>
```

```c
#include <string.h>

int main(void) {
    char perfil[20];

    printf("Digite o perfil: ");
    scanf("%19s", perfil);

    if (strcmp(perfil, "ADMIN") == 0) {
        printf("Perfil de administrador.\n");
    } else {
        printf("Perfil diferente de administrador.\n");
    }

    return 0;
}
```

A função strcmp() retorna:

Resultado	Significado
`0`	As strings são iguais
Menor que `0`	A primeira string vem antes da segunda na comparação
Maior que `0`	A primeira string vem depois da segunda na comparação

Atenção: em C, não devemos utilizar == para comparar o conteúdo de duas strings. O operador compara endereços quando aplicado aos ponteiros para seus primeiros caracteres, não o texto armazenado.

Para que serve strcasecmp()?

A função strcasecmp() é utilizada na linguagem C para comparar duas strings, ignorando a diferença entre letras maiúsculas e minúsculas. Por exemplo, ao comparar as palavras "Casa" e "CASA", a função considera que elas são iguais e retorna 0. Já a função strcmp() também compara duas strings, mas diferencia letras maiúsculas de minúsculas. Dessa forma, ao comparar "Casa" e "CASA" com strcmp(), o resultado será diferente de 0, pois as letras não são exatamente iguais. Portanto, a principal diferença entre as duas funções é que strcasecmp() ignora a diferença entre maiúsculas e minúsculas, enquanto strcmp() considera essa diferença durante a comparação.

Sintaxe:

```
strcasecmp(string1, string2);
```

6. Função strcpy() – Cópia de strings

Imagine que um sistema precise copiar o nome de um produto para outra variável.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char produto[30] = "Notebook";
    char copia[30];

    strcpy(copia, produto);
```

```
    printf("Produto original: %s\n", produto);
    printf("Produto copiado: %s\n", copia);

    return 0;
}
```

Saída esperada:

Produto original: Notebook
Produto copiado: Notebook

A função strcpy() copia os caracteres da origem para o destino, incluindo o '\0'. O vetor de destino precisa ter espaço suficiente para receber todo o conteúdo.

7. Função strcat() – Concatenação de strings

Imagine que precisamos formar uma mensagem de boas-vindas com o nome de um usuário.

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char mensagem[60] = "Bem-vindo, ";
    char nome[] = "Carlos";

    strcat(mensagem, nome);

    printf("%s!\n", mensagem);

    return 0;
}
```

Saída esperada:

Bem-vindo, Carlos!

A função strcat() acrescenta o conteúdo da segunda string ao final da primeira. O vetor que recebe a concatenação deve possuir espaço para os dois textos e o terminador '\0'.

Atividade prática para entrega

1º Questão: Uma empresa precisa desenvolver um programa para cadastrar usuários e identificar seus perfis de acesso. O sistema deverá armazenar informações básicas, validar o perfil informado e apresentar uma mensagem personalizada.

Requisitos do programa:

1. Solicitar o primeiro nome do usuário.
2. Solicitar o perfil de acesso (ADMIN ou COMUM).
3. Utilizar strlen() para informar a quantidade de caracteres do nome.
4. Utilizar strcmp() para identificar o perfil informado.
5. Utilizar strcpy() para copiar o nome para uma variável de confirmação.
6. Utilizar strcat() para montar uma mensagem de boas-vindas.
7. Exibir os dados cadastrados e a mensagem final.

Exemplo de saída:

```
===== CADASTRO DE USUARIOS =====

Digite seu nome: Mariana
Digite seu perfil: ADMIN

===== DADOS CADASTRADOS =====
Nome: Mariana
Quantidade de caracteres: 7
Perfil: Administrador

Bem-vindo(a), Mariana!
Cadastro realizado com sucesso.
```

2º Questão: Uma loja de informática deseja organizar o cadastro de seus produtos. O programa deverá receber o nome de um produto, verificar sua categoria e gerar uma descrição para identificação no estoque.

Requisitos do programa:

1. Solicitar o nome de um produto.
2. Solicitar sua categoria (HARDWARE ou PERIFERICO).
3. Utilizar strlen() para verificar a quantidade de caracteres do nome.
4. Utilizar strcmp() para identificar a categoria.

5. Utilizar strcpy() para copiar o nome do produto para outra variável.
6. Utilizar strcat() para gerar uma descrição no formato Produto: Nome.
7. Exibir um resumo do cadastro.

Exemplo de saída:

```
===== CADASTRO DE PRODUTOS =====

Nome do produto: Teclado
Categoria: PERIFERICO

===== RESUMO =====
Produto: Teclado
Caracteres: 7
Categoria: Periferico

Descricao: Produto: Teclado
Cadastro concluido!
```

3º Questão: Uma instituição de ensino deseja desenvolver um programa para identificar cursos cadastrados e gerar mensagens personalizadas para os estudantes.

Requisitos do programa:

1. Solicitar o primeiro nome do estudante.
2. Solicitar o código do curso (CC ou SI).
3. Utilizar strcmp() para identificar o curso:
    - CC: Ciência da Computação.
    - SI: Sistemas de Informação.
4. Utilizar strlen() para informar a quantidade de caracteres do nome.
5. Utilizar strcpy() para armazenar o nome em outra variável.
6. Utilizar strcat() para montar a mensagem Ola, Nome!.
7. Exibir o nome do estudante, o curso e a mensagem.

Exemplo de saída:

```
===== IDENTIFICACAO ACADEMICA =====

Nome do estudante: Lucas
Codigo do curso: CC

===== RESULTADO =====
Estudante: Lucas
Curso: Ciencia da Computacao
Caracteres no nome: 5

Ola, Lucas!
```

Orientação para o GitHub

Os alunos deverão salvar os exercícios na pasta aula-17-bibliotecas/ do repositório laboratorio-programacao-c, mantendo os códigos organizados e identificados.

Sugestão bibliográfica:

Video aula: https://www.youtube.com/watch?v=MEkrf1O_CIU

Livro: BACKES, André. *Linguagem C: Completa e Descomplicada*. 2. ed. LTC, 2018. Leitura sugerida: Capítulo 7 – Arrays de caracteres – strings.

Site: https://cppreference.com/c/header/string?utm_source=chatgpt.com